Here’s the audit report:

Balance Sheet SQL – `account_type` ambiguity audit

1. Location of Balance Sheet SQL

Item	Location
Primary RPC	`get_balance_sheet_from_trial_balance`
Definition	`supabase/migrations/169_trial_balance_canonicalization.sql`
Lines	308–336
API route	`app/api/accounting/reports/balance-sheet/route.ts` (line 87) calls `supabase.rpc("get_balance_sheet_from_trial_balance", { p_period_id })`
Secondary RPC	`get_profit_and_loss_from_trial_balance` (used for net income; same file, lines 271–299)

Legacy route `app/api/reports/balance-sheet/route.ts` (line 48) also calls `get_balance_sheet_from_trial_balance`.

---

2. Tables / sources that expose `account_type`

Source	Has `account_type`?	Where
`chart_of_accounts`	Yes	`097_chart_of_accounts_tables.sql` – column `account_type`
`get_trial_balance_from_snapshot` return	Yes	169, RETURNS TABLE (line 223) – output column `account_type`
`get_balance_sheet_from_trial_balance` return	Yes	169, RETURNS TABLE (line 315) – output column `account_type`
`get_profit_and_loss_from_trial_balance` return	Yes	169, RETURNS TABLE (line 277) – output column `account_type`
`accounts`	No	Has `type` only (043, 138, 140, etc.)
`trial_balance_snapshots`	No	Has `snapshot_data` (JSONB) only

---

3. Unqualified `account_type` references in Balance Sheet-related SQL

All in `169_trial_balance_canonicalization.sql`:

File	Line	Snippet	Context
`169_trial_balance_canonicalization.sql`	287	`WHERE account_type IN ('income', 'expense')`	P&L: `FOR ... IN SELECT * FROM get_trial_balance_from_snapshot(p_period_id)`
`169_trial_balance_canonicalization.sql`	325	`WHERE account_type IN ('asset', 'liability', 'equity')`	Balance sheet: `FOR ... IN SELECT * FROM get_trial_balance_from_snapshot(p_period_id)`

| `169_trial_balance_canonicalization.sql` | 397 | `WHERE account_type IN ('income', 'expense')` | `validate_statement_reconciliation`: same pattern |
| `169_trial_balance_canonicalization.sql` | 410 | `WHERE account_type IN ('asset', 'liability', 'equity')` | `validate_statement_reconciliation`: same pattern |

Qualified uses (record fields): 293, 331, 399, 412, 414 (`trial_balance_row.account_type`, `trial_balance_account.account_type`).

---

4. Exact SQL snippets (Balance Sheet)

`get_balance_sheet_from_trial_balance` (169, 322–334):

```sql
  FOR trial_balance_row IN
    SELECT *
    FROM get_trial_balance_from_snapshot(p_period_id)
    WHERE account_type IN ('asset', 'liability', 'equity')
  LOOP
    RETURN QUERY SELECT
      trial_balance_row.account_id,
      trial_balance_row.account_code,
      trial_balance_row.account_name,
      trial_balance_row.account_type,
      trial_balance_row.closing_balance;
  END LOOP;
```

`get_profit_and_loss_from_trial_balance` (169, 285–296) uses the same pattern with `WHERE account_type IN ('income', 'expense')`.
`validate_statement_reconciliation` (169, 394–418) uses both patterns.

---

5. Tables involved and who “owns” `account_type`

Query	FROM clause	Tables / sources in scope	Who has `account_type`
Balance Sheet loop	`FROM get_trial_balance_from_snapshot(p_period_id)`	SRF result only	**SRF output** (single source)
P&L loop	Same	Same	**SRF output**
`validate_statement_reconciliation`	Same	Same	**SRF output**

`get_trial_balance_from_snapshot` (169, 216–261) reads from `trial_balance_snapshots` and iterates over `jsonb_array_elements(snapshot_record.snapshot_data)`. It does not join any base table. Its return set has a single `account_type` column.

---

6. Which tables would introduce ambiguity?

- **In the current Balance Sheet SQL:** The only source in the FROM clause is `get_trial_balance_from_snapshot(...)`. There is no join. So **from the codebase, no second table with `account_type` is in scope** for those queries.
- **`chart_of_accounts`** has `account_type` but is **not** used in:
  - `get_balance_sheet_from_trial_balance`
  - `get_trial_balance_from_snapshot`
  - `get_profit_and_loss_from_trial_balance`
  - `validate_statement_reconciliation`
  So it does not currently participate in Balance Sheet SQL.

Conclusion: The Balance Sheet SQL uses a single table (the trial balance SRF result) and unqualified `account_type` only in WHERE clauses against that one source. Under the current migrations, that alone would not cause “column reference 'account_type' is ambiguous.”

---

7. Summary

Item	Value
File	`supabase/migrations/169_trial_balance_canonicalization.sql`
Relevant lines	285–296 (P&L), 322–334 (Balance Sheet), 394–418 (`validate_statement_reconciliation`)
Exact unqualified uses	`WHERE account_type IN (...)` in FOR-loop SELECTs over `get_trial_balance_from_snapshot(p_period_id)`
Tables in those FROM clauses	Only the set-returning function result
`account_type` ownership	Solely the `get_trial_balance_from_snapshot` return columns

Verdict: The Balance Sheet runs `get_balance_sheet_from_trial_balance` (and, for net income, `get_profit_and_loss_from_trial_balance`). Both use `FROM get_trial_balance_from_snapshot(p_period_id)` only, with unqualified `account_type` in the WHERE. No second table with `account_type` is joined in that SQL. If you actually see “column reference 'account_type' is ambiguous,” it may be due to (a) a different migration or deployed version that adds a join (e.g. to `chart_of_accounts` or another `account_type`-bearing table), (b) a different report or code path that shares this SQL, or (c) environment-specific schema or PostgreSQL behavior. The minimal fix would be to qualify `account_type` (e.g. by aliasing the SRF and using `alias.account_type`) in those WHERE clauses, but per your rules no fix has been applied here.

Ledger Imbalance Audit Report

Objective

Determine **why** the Trial Balance reports "Ledger is Not Balanced" by identifying all journal entries where `sum(debit) ≠ sum(credit)` and classifying each by root cause.

Scope

- **Tables:** `journal_entries`, `journal_entry_lines`, and related `invoices`,

`payments`
- **Rules:** Read-only. No deletes, updates, inserts, or migrations.

---

1. How "Ledger is Not Balanced" Arises

- The Trial Balance UI shows **"⚠ Ledger is Not Balanced"** when `totalDebits ≠ totalCredits` for the period (`app/trial-balance/page.tsx`, `app/accounting/reports/trial-balance`).
- Trial balance data comes from `get_trial_balance_from_snapshot` → `trial_balance_snapshots`, which aggregates `journal_entry_lines` (and opening balances) per account.
- **Global** `sum(debits) = sum(credits)` holds only if **every** journal entry is balanced. Any entry with `sum(debit) ≠ sum(credit)` contributes to an overall imbalance.
- `generate_trial_balance` **raises** if the period is imbalanced, so a persisted snapshot implies the ledger was balanced **at generation time**. Imbalance can still appear if:
  - The UI uses a different data path (e.g. legacy report), or
  - There is rounding/aggregation differences, or
  - The snapshot is stale and new imbalanced data was introduced (e.g. pre-trigger legacy data, or a path that bypasses the trigger).

---

2. Identification of Imbalanced Entries

We use the same logic as `validate_period_ready_for_close` (migration 167):

```sql
GROUP BY je.id
HAVING ABS(COALESCE(SUM(jel.debit), 0) - COALESCE(SUM(jel.credit), 0)) > 0.01
```

- **Tolerance:** `0.01` (matches `enforce_double_entry_balance_statement` in migration 188).
- **Result:** All `journal_entry_id` values where `|sum(debit) - sum(credit)| > 0.01`.

---

3. Output: Table of Offending Entries

Column	Description
`journal_entry_id`	UUID of the imbalanced journal entry
`reference_type`	`invoice`, `payment`, `sale`, `adjustment`, `manual`, etc.
`reference_id`	UUID of linked entity (invoice, payment, etc.), or NULL
`created_at`	When the journal entry was created
`business_id`	Business owning the entry
`date`	Journal entry date
`description`	Entry description

`total_debit`	Sum of `journal_entry_lines.debit` for this entry
`total_credit`	Sum of `journal_entry_lines.credit` for this entry
`imbalance_amount`	`|total_debit - total_credit|`
`line_count`	Number of lines in the entry
`root_cause_category`	One of: `interrupted posting`, `tax_lines missing`, `legacy data`, `manual entry`, `unknown`

---

4. Root-Cause Classification

Each imbalanced entry is assigned a **single** root-cause category using this precedence:

Category	Condition
interrupted posting	`line_count = 1` (single-line entry cannot balance)
tax_lines missing	`reference_type = 'invoice'`, linked invoice exists, `(total_tax > 0 OR apply_taxes)`, and `tax_lines` is NULL, empty, or not usable (no `tax_lines` key or empty array)
legacy data	`created_at < '2024-01-01'`
manual entry	`reference_type IN ('manual', 'adjustment')`
unknown	None of the above

- **Legacy:** Entries (or linked invoices/payments) created before a cutoff; may predate balance enforcement or tax normalization.
- **tax_lines missing:** Invoices that have tax (or `apply_taxes`) but lack ledger-ready `tax_lines`; linked postings can be wrong or partial.
- **Interrupted posting:** Single-line entries are never balanced; suggests aborted or partial posting.
- **Manual entry:** Adjustments or manual journals; may indicate user error or a bug in the adjustment flow.

---

5. Script Location and Usage

File: `scripts/audit-ledger-imbalance.sql`

Usage:

1. Open Supabase SQL Editor (or `psql`).
2. Run the script. It contains **only** `SELECT` (and commented optional summary). No `INSERT`/`UPDATE`/`DELETE`.
3. Use the result set as the **table of offending journal_entry_ids** with imbalance amount and root-cause category per entry.

Optional: Uncomment and run the second block to get **summary counts** by `root_cause_category` and total imbalance per category.

---

6. Links to Invoices and Payments

- **Invoices:** `journal_entries.reference_type = 'invoice'` and `journal_entries.reference_id = invoices.id`. The script left-joins `invoices` to detect `tax_lines` missing and `invoice_created_at` for context.
- **Payments:** `reference_type = 'payment'` and `reference_id = payments.id`. The script left-joins `payments` for context; classification does not use payment-specific logic beyond presence.

---

7. Related Code

- **Balance enforcement:** `enforce_double_entry_balance_statement` (migration 188), `post_journal_entry` (batch insert).
- **Trial balance:** `generate_trial_balance`, `get_trial_balance_from_snapshot` (migration 169).
- **Period close check:** `validate_period_ready_for_close` (migration 167) counts unbalanced entries in the period.

---

8. Summary

- **What:** Read-only audit of all journal entries with `sum(debit) ≠ sum(credit)`.
- **Where:** `scripts/audit-ledger-imbalance.sql`.
- **Output:** Table of offending `journal_entry_id`s with `reference_type`, `reference_id`, `created_at`, imbalance amount, line count, and root-cause category.
- **Categories:** `interrupted posting` | `tax_lines missing` | `legacy data` | `manual entry` | `unknown`.

Run the script, then use the results to prioritise fixes (e.g. legacy corrections, tax_lines backfill, or adjustment-path fixes) and to clear "Ledger is Not Balanced" once all imbalanced entries are resolved.

Financial Reports – Draft Invoices in Outstanding (Audit)

Objective

Identify **where** draft invoices enter reporting aggregates on the Financial Reports page, specifically the **Outstanding** metric.

Rules: Read-only. No filters added. No logic changes.

---

1. Location of Outstanding Calculation

Item	Value
File	`app/reports/page.tsx`
Outstanding logic	Lines 59–61

| **Data source** | Direct Supabase query to `invoices` (lines 47–52) |

---

2. Query That Feeds Outstanding

File: `app/reports/page.tsx`
Lines: 47–52

```ts
const { data: invoicesData } = await supabase
  .from("invoices")
  .select("id, total, status")
  .eq("business_id", business.id)
  .is("deleted_at", null)
```

- **Filters applied:** `business_id`, `deleted_at IS NULL` only.
- **No status filter:** All non-deleted invoices (any status) are fetched.

---

3. Outstanding Calculation Snippet

File: `app/reports/page.tsx`
Lines: 59–61

```ts
const outstandingAmount = invoicesData
  .filter((inv: any) => inv.status !== "paid" && inv.status !== "cancelled")
  .reduce((sum: number, inv: any) => sum + Number(inv.total || 0), 0)
```

- **Filter:** `status !== "paid"` and `status !== "cancelled"`.
- **Aggregation:** Sum of `invoice.total` over the filtered list.

---

4. Invoice Statuses Included in Outstanding

Outstanding includes every invoice whose status is **not** `paid` and **not** `cancelled`. That implies:

Status	Included in Outstanding?
`draft`	**Yes**
`sent`	Yes
`partially_paid`	Yes
`overdue`	Yes
`void`	Yes
`converted`	Yes
`paid`	No
`cancelled`	No

So **draft** (and void, converted) are included.

---

5. Missing or Incorrect Status Filters

- **Missing:** There is **no** exclusion of `draft` (or of `void` / `converted`).
- **Effective filter:** Only `paid` and `cancelled` are excluded.
- **Gap:** Drafts are treated as outstanding because they are “not paid and not cancelled.”

---

6. Intentional vs Accidental Inclusion of Drafts

Conclusion: accidental.

Evidence:

1. **Invoices page** (`app/invoices/page.tsx`, 441–460) explicitly excludes drafts from outstanding:
   - Comment: *"Draft invoices are NOT financial documents and cannot be outstanding"*.
   - Filter: `inv.status !== "draft"` and only `sent` | `overdue` | `partially_paid` for outstanding.
2. **Aging report API** (`app/api/reports/aging/route.ts`, 51–72):
   - Comment: *"Exclude draft invoices - drafts are NOT financial documents and cannot be outstanding"*.
   - Query: `.in("status", ["sent", "partially_paid", "overdue"])` and `.neq("status", "draft")`.
3. **Financial Reports page** has no equivalent exclusion and no comment about drafts. Its Outstanding logic only excludes `paid` and `cancelled`, so drafts are included by omission.

Drafts are therefore included **accidentally** due to a missing status filter on the Financial Reports page, not by design.

---

7. Summary

Item	Detail
File	`app/reports/page.tsx`
Query (data)	47–52: `invoices` with `business_id`, `deleted_at IS NULL`; no status filter
Outstanding (agg)	59–61: `filter(status !== "paid" && status !== "cancelled")` then `sum(total)`
Statuses included	`draft`, `sent`, `partially_paid`, `overdue`, `void`, `converted`
Statuses excluded	`paid`, `cancelled`
Missing filter	No exclusion of `draft` (or `void` / `converted`)
Why drafts are included	Filter only removes paid/cancelled; drafts match

“not paid and not cancelled” |
| **Intentional?** | No. Invoices page and Aging API exclude drafts; Reports page does not. |

---

8. Root Cause

Outstanding on the Financial Reports page uses a **status filter that excludes only `paid` and `cancelled`**. Because **draft** is not excluded, draft invoices are included in Outstanding. This is a **missing status filter**, not an intentional choice.

Financial Consistency Audit – Operational vs Accrual Metrics

Objective

Ensure operational and accounting metrics do **not** contradict across Dashboard, Financial Reports, and VAT page.

Rules: Read-only.

---

1. Page Classification: Operational vs Accrual

Page	Basis	Source
Dashboard	**Mixed**	Revenue & Outstanding: **accrual** (ledger). Collected & Total Expenses: **operational** (invoices/payments/expenses).
Financial Reports	**Operational**	All summary metrics from `invoices` table (no ledger).
VAT (reports/vat)	**Accrual**	VAT Control from `journal_entry_lines` (account 2100) only.

---

2. Definitions by Page

Dashboard (`app/dashboard/page.tsx`)

Metric	Definition	Source	Scope
Total Revenue	Sum of credits to Revenue account (4000)	`journal_entry_lines` → account 4000	All-time, no date filter
Collected	Sum of `payments.amount`	`payments`	This month only
Outstanding	AR ledger balance: max(0, Σ debit – Σ credit) for account 1200	`journal_entry_lines` → account 1200	All-time
Total Expenses	Sum of `expenses.total`	`expenses`	All-time
Tax base	Not shown on dashboard	—	—

- **Revenue:** Accrual, net (ledger 4000 = revenue account; VAT in 2100).
- **Outstanding:** Accrual (AR balance). Overdue *count* uses operational

invoice-level data; the **Outstanding** KPI *amount* is ledger-based.

Financial Reports (`app/reports/page.tsx`)

Metric	Definition	Source	Scope
Total Revenue	Sum of `invoice.total` where `status === "paid"`	`invoices`	All-time
Outstanding	Sum of `invoice.total` where `status !== "paid"` and `status !== "cancelled"`	`invoices`	All-time
Total Invoices	Count of non-deleted invoices	`invoices`	All-time
Paid Invoices	Count where `status === "paid"`	`invoices`	All-time
Tax base	Not shown	—	—

- **Revenue:** Operational, **gross** (invoice total includes tax), **cash** (paid only).
- **Outstanding:** Operational. Raw sum of totals of non-paid, non-cancelled invoices. **No** subtraction of payments or credit notes. **Includes drafts.**

VAT Page (`app/reports/vat/page.tsx` → `/api/reports/vat-control`)

Metric	Definition	Source	Scope
Opening balance	Σ(credit – debit) for account 2100 before period start	`journal_entry_lines` → 2100	Before `start_date`
VAT collected	Σ credits to 2100 in period	`journal_entry_lines` → 2100	`start_date`–`end_date`
VAT reversed	Σ debits to 2100 in period	`journal_entry_lines` → 2100	`start_date`–`end_date`
Closing balance	opening + VAT collected – VAT reversed	Derived	After `end_date`
Tax base	Not shown	—	—

- **VAT:** Accrual. VAT liability movement only. No explicit “tax base” (taxable sales) on the page.

---

3. Table: Page → Definition Used

Page	Revenue	Outstanding	Tax base
Dashboard	Ledger 4000 credits (accrual, net, all-time)	AR 1200 balance (accrual, all-time)	—
Financial Reports	Sum(paid `invoice.total`) (operational, gross, cash)	Sum(non-paid, non-cancelled `invoice.total`); includes drafts; no payments/credits	—
VAT	—	—	Not shown; VAT = ledger 2100 (accrual)

---

4. Semantic Mismatches and Inconsistencies

4.1 Revenue

Issue	Where	Detail
Different basis	Dashboard vs Financial Reports	Dashboard “Total Revenue” = **accrual, net** (ledger 4000). Financial Reports “Total Revenue” = **cash, gross** (paid `invoice.total`). Same label, different meaning.
Net vs gross	Dashboard vs Financial Reports	Dashboard excludes VAT (net). Financial Reports uses `invoice.total` (gross). Numbers can differ materially.
Timing	Dashboard vs Financial Reports	Dashboard: accrual (all-time revenue recognition). Financial Reports: paid-only (cash timing).

4.2 Outstanding

Issue	Where	Detail
Accrual vs operational	Dashboard vs Financial Reports	Dashboard = AR ledger (accrual). Financial Reports = sum of invoice totals (operational) with **no** payments/credits.
Definition of “outstanding”	Financial Reports	Not “total – payments – credits” per invoice. It is sum of **full** `invoice.total` for non-paid, non-cancelled. Overstates true outstanding.
Drafts included	Financial Reports	Drafts included (status ≠ paid, cancelled). Drafts are not financial; they should not be outstanding.
Void / converted	Financial Reports	Same filter includes `void` and `converted`, which may not represent collectible outstanding.

4.3 Tax base

Issue	Where	Detail
Not exposed	All three	No page shows “tax base” or “taxable sales” explicitly. VAT page shows only VAT liability (2100) movement.

4.4 Other

Issue	Where	Detail
Expenses	Dashboard only	`expenses` table (operational). Financial Reports and VAT do not show expense metrics.
Collected	Dashboard only	“Collected” (cash this month) exists only on Dashboard. Financial Reports has no equivalent.

---

5. Summary of Inconsistencies

1. **“Total Revenue”** means **accrual net** on Dashboard and **cash gross** on Financial Reports → **contradiction**.
2. **“Outstanding”** means **AR ledger** on Dashboard and **sum of unpaid invoice totals (incl. drafts, no payments/credits)** on Financial Reports → **contradiction** and **incorrect** definition on Financial Reports.
3. **Financial Reports Outstanding** includes **drafts** and does **not** subtract payments or credit notes → **semantic error** (already noted in

draft-outstanding audit).
4. **VAT page** is accrual-only (ledger 2100); **no tax base** reported. Dashboard and Financial Reports do not show VAT or tax base.

---

6. Page → Definition (Quick Reference)

Page	Revenue	Outstanding	Tax base
Dashboard	Ledger 4000 (accrual, net)	AR 1200 (accrual)	—
Financial Reports	Paid `invoice.total` (operational, gross)	Sum unpaid `invoice.total` (operational; includes drafts; no payments/credits)	—
VAT	—	—	Not shown (VAT = 2100 accrual)

These differences are **semantic mismatches**: same or similar labels used with different bases, definitions, and inclusions across pages.

Customer Statement – `currencySymbol is not defined` (Audit)

Objective

Determine where `currencySymbol` is expected to come from and why the Customer Statement page crashes.

Rules: Read-only. No variable declarations. No fixes.

---

1. All References to `currencySymbol`

Line	Context
109	WhatsApp message template: `` `...Total Outstanding: ${currencySymbol \|\| ""}${summary?.totalOutstanding.toFixed(2) || "0.00"}.` ``
258	Summary card: `{currencySymbol || ""}{summary.totalInvoiced.toFixed(2)}`
262	Summary card: `{currencySymbol || ""}{summary.totalPaid.toFixed(2)}`
267	Summary card: `-{currencySymbol || ""}{summary.totalCredits.toFixed(2)}`
272	Summary card: `{currencySymbol || ""}{summary.totalOutstanding.toFixed(2)}`
276	Summary card: `{currencySymbol || ""}{summary.totalOverdue.toFixed(2)}`
304	Invoice total: `{currencySymbol || ""}{Number(invoice.total).toFixed(2)}`
306	Invoice balance: `Balance: {currencySymbol || ""}{balance.toFixed(2)}`
319	Payment amount: `{currencySymbol || ""}{Number(payment.amount).toFixed(2)}`
333	Credit note total: `-{currencySymbol ||

""}{Number(creditNote.total).toFixed(2)}` |

Total: 10 references. All use `currencySymbol || ""`; none define it.

---

2. Where `currencySymbol` SHOULD Be Defined

Source	Details
Hook	`useBusinessCurrency()` from `@/lib/hooks/useBusinessCurrency`
Hook provides	`currencySymbol: string | null` (from `getCurrencySymbol(business.default_currency)`)
Ultimate source	Business settings: `business.default_currency` → `getCurrencySymbol(code)`

Expected pattern (as used elsewhere):
- `app/customers/[id]/360/page.tsx` (line 52): `const { currencyCode, currencySymbol } = useBusinessCurrency()`
- `app/estimates/[id]/view/page.tsx` (line 50): `const { currencySymbol } = useBusinessCurrency()`

The statement page **imports** `useBusinessCurrency` (line 6) but **does not call it**. No destructuring, no `currencySymbol` in scope.

---

3. Existence in File and Imports

Item	Present?	Location
Import	Yes	Line 6: `import { useBusinessCurrency } from "@/lib/hooks/useBusinessCurrency"`
Hook invocation	**No**	No `useBusinessCurrency()` call in the component
State / local var	**No**	No `useState` or `let`/`const` for `currencySymbol`
Prop	No	Component has no props; `currencySymbol` is not passed in
Context	No	No React context consumed for currency

Conclusion: `currencySymbol` is used but never declared or derived. The hook that would supply it is imported but never used.

---

4. Root Cause Classification

Classification	Applicable?	Reason
Missing import	No	`useBusinessCurrency` is imported.
Missing derivation	**Yes**	Hook is imported but **never called**. `currencySymbol` must be derived via `const { currencySymbol } = useBusinessCurrency()`. That call is absent.
Removed refactor artifact	Partially	Usages of `currencySymbol` exist

(likely added when replacing hardcoded "₵"/"GHS"). The **definition** (hook call) was either never added or was removed in a refactor. The variable was never introduced in this file. |

Verdict: missing derivation. The hook exists and is imported; the component simply never invokes it, so `currencySymbol` is never defined. The crash is a **ReferenceError** when any of the 10 references is evaluated.

---

5. Summary

Item	Value
File	`app/customers/[id]/statement/page.tsx`
References	Lines 109, 258, 262, 267, 272, 276, 304, 306, 319, 333
Expected source	`useBusinessCurrency()` → `currencySymbol` from business `default_currency`
Root cause	**Missing derivation**: hook imported (line 6) but never called; no other definition of `currencySymbol`.

VAT Report Logic – January 2026 Correctness Audit

Objective

Confirm VAT page correctness for January 2026.

Scope: VAT report logic only. **Rules:** Read-only.

---

1. VAT Reporting Surfaces in Scope

Surface	Route / API	Purpose
VAT page	`/reports/vat` → `GET /api/reports/vat-control`	VAT Control (account 2100)
VAT Returns calculate	`POST /api/vat-returns/calculate`	Ghana VAT return calculation
VAT Returns monthly	`GET /api/vat-returns/monthly`	Monthly VAT return summaries

Out of scope: Tax summary (deprecated 410), accounting VAT export (separate export flow).

---

2. Output VAT Calculation Sources

2.1 VAT page (VAT Control)

Item	Source	Details

| **Output VAT** | `journal_entry_lines` | Sum of **credits** to account **2100** in period. Label: "VAT Collected." |
| **Filter** | `journal_entries.date` in `[start_date, end_date]` | Date range from UI. |

Verified: Output VAT = period credits to 2100. Source is ledger only.

2.2 VAT Returns calculate

Item	Source	Details
Output VAT	`invoices`, `credit_notes`	Invoices: `status = 'paid'`, `apply_taxes = true`, `issue_date` in period. Credit notes `status = 'applied'`, `date` in period.
Formula	`Σ(inv.vat) – Σ(cn.vat)`	Excludes credit-note VAT from output.

Verified: Output VAT from paid, tax-applied invoices minus applied credit notes. Correct.

2.3 VAT Returns monthly

Item	Source	Details
Output VAT	`invoices` (and credit notes if used)	Per-month grouping by `issue_date`. `output_vat` = Σ `inv.vat` for month.

Verified: Consistent with calculate; output from invoices.

---

3. Input VAT Calculation Sources

3.1 VAT page (VAT Control)

Item	Source	Details
Input VAT	`journal_entry_lines`	Sum of **debits** to account **2100** in period. Label: "VAT Reversed."

Verified: Input VAT = period debits to 2100. Ledger-only.

3.2 VAT Returns calculate

Item	Source	Details
Input VAT	`expenses`, `bills`	Expenses: `date` in period, **filtered to `vat > 0` only**. Bills: `issue_date` in period, with tax.
Comment	"Only VAT is deductible"	Lines 129, 186–187, 206–208.
Formula	`Σ(exp.vat) + Σ(bill.vat)`	No NHIL/GETFund/COVID in input VAT.

Verified: Input VAT from expenses and bills; only VAT used. Correct.

3.3 VAT Returns monthly

Item	Source	Details
Input VAT	`expenses`, `bills`	`input_vat` = Σ `exp.vat` + Σ `bill.vat` per month.

Verified: Same logic as calculate; input from expenses and bills.

---

4. NHIL, GETFund, COVID: Reporting vs Deductible

4.1 Included in reporting?

Surface	NHIL	GETFund	COVID	Notes
VAT page	**No**	**No**	**No**	Uses **only** account 2100. No separate levies.
VAT Returns calculate	**Yes**	**Yes**	**Yes**	`total_output_nhil/getfund/covid`, `total_input_*`, `total_output_tax`, `total_input_tax`.
VAT Returns monthly	**Yes**	**Yes**	**Yes**	`output_nhil/getfund/covid`, `input_*`, all in `monthlyReturns`.

4.2 Excluded from deductible VAT?

Surface	Logic	Verified
VAT Returns calculate	"NHIL, GETFund, COVID on expenses are ignored for VAT return purposes." Net = `output VAT – input VAT` only.	**Yes**
VAT Returns monthly	"NHIL, GETFund, COVID are levies (report-only, not deductible)." `net_vat = output_vat – input_vat`.	**Yes**
VAT page	No deductibility logic; single account 2100.	N/A

Conclusion:
- **VAT page:** NHIL, GETFund, COVID are **not** included in reporting (only 2100).
- **VAT Returns:** NHIL, GETFund, COVID **are** included in reporting and **are** excluded from deductible VAT.

---

5. Net VAT Payable Formula

5.1 VAT page (VAT Control)

Item	Formula	Notes
Closing balance	`opening + vat_collected – vat_reversed`	Equals opening + credits – debits.
Invariant	`opening + credits – debits = closing`	Checked in UI.

No explicit "Net VAT Payable" label; closing is the liability balance. Formula

itself is correct.

5.2 VAT Returns calculate

Item	Formula	Location
Net VAT Payable	`max(totalOutputVat – totalInputVat, 0)`	Line 214.
Net VAT Refund	`max(totalInputVat – totalOutputVat, 0)`	Line 215.

Verified: Net VAT = output VAT – input VAT; levies not used. Correct.

5.3 VAT Returns monthly

Item	Formula	Location
Net VAT	`output_vat – input_vat`	Line 211.

Verified: Same concept; levies not in net. Correct.

---

6. January 2026 Specifics

Check	Result
COVID removed from 2026	Ledger extraction (`extract_tax_return_from_ledger`, migration 093): **skips** account 2130 (COVID) for `p_start_date >= '2026-01-01'`.
VAT page	Uses 2100 only. Posting: VAT→2100; NHIL/GETFund/COVID→2110/2120/2130. Jan 2026 VAT Control unaffected.
VAT Returns	No date-based COVID filter in calculate/monthly. Jan 2026 returns use invoices **issued** in Jan 2026; tax engine does not add COVID for 2026, so `covid` = 0.

Verified: January 2026 handling is correct.

---

7. Discrepancies

#	Surface	Issue
1	**VAT page**	NHIL, GETFund, COVID are **not** included in reporting. The page uses only account 2100. If "VAT page" must report levies, this is a **gap**.
2	**VAT page**	No explicit "Net VAT Payable" metric; only closing balance. Semantically equivalent but not explicitly labeled.

No discrepancies found in **VAT Returns** logic (calculate / monthly).

---

8. Summary

Surface	Output VAT	Input VAT	NHIL/GETFund/COVID in reporting	Excluded from deductible	Net VAT formula	Jan 2026
VAT page	✓ Credits 2100	✓ Debits 2100	**✗ Not included**	N/A	✓ closing = opening + credits – debits	✓
VAT Returns calculate	✓	✓	✓	✓	✓	✓
VAT Returns monthly	✓	✓	✓	✓	✓	✓

---

9. Confirmation

VAT Returns logic (calculate, monthly): CORRECT

- Output VAT: invoices (and credit notes where applicable); input VAT: expenses + bills (VAT only).
- NHIL, GETFund, COVID included in reporting, excluded from deductible.
- Net VAT Payable = output VAT – input VAT.
- January 2026: COVID correctly excluded (ledger extraction); VAT Returns rely on 2026 invoice data.

VAT page (VAT Control): NOT fully correct for the stated criteria.

- Output/input VAT sources and net-style formula (closing balance) are correct.

- **NHIL, GETFund, COVID are not included in reporting** on the VAT page (2100 only).
- If the audit scope requires the **VAT page** to include levies and explicit Net VAT Payable, then:

  Overall: NOT — due to VAT page gap on levies (and optional labelling of Net VAT Payable).

If the scope is **VAT Returns only**, then **CORRECT**.

---

10. Root Cause of VAT Page Gap

The VAT page (`/reports/vat`) calls only `GET /api/reports/vat-control`, which reads **account 2100**. Levies (2110, 2120, 2130) are never queried or displayed. Including them would require either using `extract_tax_return_from_ledger` (or equivalent) or extending vat-control to aggregate 2100–2130 and surface them on the VAT page.